

JavaScript and React.js Workshop Material

A Comprehensive Guide for Beginners

This document summarizes the key concepts and code examples from the workshop.

Contents

1	Introduction	2
2	JavaScript Basics	2
2.1	Linking JavaScript with HTML	2
2.2	Selecting DOM Elements	3
2.3	Event Listeners	3
2.4	Handling Button Click Events	3
2.5	Types of Functions	4
3	React.js Basics	4
3.1	Setup Instructions	4
3.2	Counter with Class Components	5
3.3	Counter with Functional Components	5
3.4	Button Click Events	6
3.5	Conditional Rendering	6
3.6	String Literals	7
4	Advanced React Concepts	8
4.1	useState Hook	8
4.2	useEffect Hook	8
4.3	Props for Data Sharing	9
5	Additional Resources	11
6	Conclusion	11

1 Introduction

This document provides a comprehensive summary of the JavaScript and React.js workshop. It covers essential JavaScript concepts for web interactivity and foundational React.js techniques for building modern web applications. The material is designed for college students with basic HTML and CSS knowledge, aiming to equip them with practical skills in front-end development.

Workshop Objectives:

- Understand JavaScript DOM manipulation, events, and functions.
- Learn React.js components, hooks, and data sharing.
- Apply concepts through hands-on code examples.

Prerequisites:

- Basic understanding of HTML and CSS.
- Node.js installed for React.js setup (<https://nodejs.org>).

2 JavaScript Basics

2.1 Linking JavaScript with HTML

To connect a JavaScript file to an HTML page, use the `<script>` tag with the `src` attribute. Place it at the bottom of the `<body>` for optimal loading.

Listing 1: index.html

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale
6     =1.0">
7   <title>JavaScript Demo</title>
8 </head>
9 <body>
10   <h1>Welcome to JavaScript Learning</h1>
11   <div id="container">
12     <p class="demo-text">This is a paragraph</p>
13     <button id="myButton">Click Me!</button>
14   </div>
15   <script src="script.js"></script>
16 </body>
</html>
```

Listing 2: script.js

```
1 console.log("JavaScript file linked successfully!");
```

Note: Check the browser console (F12) to verify the connection.

2.2 Selecting DOM Elements

JavaScript offers multiple methods to select elements for manipulation.

Listing 3: script.js (Element Selection)

```
1 const button = document.getElementById('myButton');
2 console.log("Button element:", button);
3
4 const demoTexts = document.getElementsByClassName('demo-text');
5 console.log("Elements with class 'demo-text':", demoTexts);
6
7 const paragraphs = document.getElementsByTagName('p');
8 console.log("All paragraphs:", paragraphs);
9
10 const firstPara = document.querySelector('p');
11 console.log("First paragraph:", firstPara);
12
13 const allParagraphs = document.querySelectorAll('p');
14 console.log("All paragraphs using querySelectorAll:", allParagraphs);
```

Note: `querySelector` is versatile, using CSS-like selectors.

2.3 Event Listeners

Event listeners enable interactive features by responding to user actions.

Listing 4: script.js (Event Listeners)

```
1 const button = document.getElementById('myButton');
2
3 button.addEventListener('click', function() {
4     console.log("Button clicked using addEventListener!");
5 });
6
7 button.addEventListener('mouseover', function() {
8     console.log("Mouse over the button!");
9 });
10
11 button.addEventListener('mouseout', function() {
12     console.log("Mouse left the button!");
13 });
14
15 function handleDoubleClick() {
16     console.log("Button double-clicked!");
17 }
18 button.addEventListener('dblclick', handleDoubleClick);
```

Note: `addEventListener` supports multiple listeners, unlike `onclick`.

2.4 Handling Button Click Events

Click events can modify the DOM dynamically, such as updating text or styles.

Listing 5: script.js (Click Events)

```
1 const clickButton = document.getElementById('myButton');
2
3 clickButton.addEventListener('click', function() {
4     clickButton.textContent = "Clicked!";
5     clickButton.style.backgroundColor = "green";
6     clickButton.style.color = "white";
7
8     const container = document.getElementById('container');
9     const newPara = document.createElement('p');
10    newPara.textContent = "Button was clicked!";
11    container.appendChild(newPara);
12 });
```

Note: Use `createElement` and `appendChild` for dynamic content.

2.5 Types of Functions

JavaScript supports three function types: declaration, expression, and arrow.

Listing 6: script.js (Functions)

```
1 function addNumbers(a, b) {
2     return a + b;
3 }
4
5 const multiplyNumbers = function(a, b) {
6     return a * b;
7 };
8
9 const divideNumbers = (a, b) => {
10     return a / b;
11 };
12
13 const squareNumber = num => num * num;
14
15 console.log("Function Declaration - Add:", addNumbers(5, 3));
16 console.log("Function Expression - Multiply:", multiplyNumbers(5, 3));
17 console.log("Arrow Function - Divide:", divideNumbers(10, 2));
18 console.log("Arrow Function - Square:", squareNumber(4));
```

Note: Arrow functions are concise and ideal for callbacks.

3 React.js Basics

3.1 Setup Instructions

Install Node.js and create a React app:

```
1 npm create vite@latest react-learning
2 cd react-learning
```

```
3 npm install
4 npm run dev
```

Access at <http://localhost:5173>.

3.2 Counter with Class Components

Class components manage state with `this.state`.

Listing 7: CounterClass.js

```
1 import React, { Component } from 'react';
2
3 class CounterClass extends Component {
4   constructor(props) {
5     super(props);
6     this.state = {
7       count: 0
8     };
9   }
10
11   increment = () => {
12     this.setState({ count: this.state.count + 1 });
13   }
14
15   decrement = () => {
16     this.setState({ count: this.state.count - 1 });
17   }
18
19   render() {
20     return (
21       <div style={{ padding: '20px', border: '1px solid #ccc',
22         margin: '10px' }}>
23         <h2>Class Component Counter</h2>
24         <p>Count: {this.state.count}</p>
25         <button onClick={this.increment}>+</button>
26         <button onClick={this.decrement}>-</button>
27       </div>
28     );
29   }
30 }
export default CounterClass;
```

3.3 Counter with Functional Components

Functional components use the `useState` hook.

Listing 8: CounterFunctional.js

```
1 import React from 'react';
2
3 function CounterFunctional() {
4   const [count, setCount] = React.useState(0);
```

```
5
6   return (
7     <div style={{ padding: '20px', border: '1px solid #ccc',
8       margin: '10px' }}>
9       <h2>Functional Component Counter</h2>
10      <p>Count: {count}</p>
11      <button onClick={() => setCount(count + 1)}>+</button>
12      <button onClick={() => setCount(count - 1)}>-</button>
13    </div>
14  );
15 }
export default CounterFunctional;
```

3.4 Button Click Events

React handles events with `onClick`.

Listing 9: ClickHandler.js

```
1 import React from 'react';
2
3 function ClickHandler() {
4   const handleClick = (message) => {
5     alert(message);
6   };
7
8   const handleInputClick = (event) => {
9     console.log('Button clicked!', event.target);
10  };
11
12  return (
13    <div style={{ padding: '20px', border: '1px solid #ccc',
14      margin: '10px' }}>
15      <h2>Button Click Events</h2>
16      <button onClick={() => alert('Hello!')}>
17        Simple Click
18      </button>
19      <button onClick={() => handleClick('Button clicked!')}>
20        Click with Parameter
21      </button>
22      <button onClick={handleInputClick}>
23        Event Object Demo
24      </button>
25    </div>
26  );
27 }
export default ClickHandler;
```

3.5 Conditional Rendering

Render elements based on state or conditions.

Listing 10: ConditionalRender.js

```
1 import React, { useState } from 'react';
2
3 function ConditionalRender() {
4   const [isLoggedIn, setIsLoggedIn] = useState(false);
5   const [score, setScore] = useState(75);
6
7   return (
8     <div style={{ padding: '20px', border: '1px solid #ccc',
9       margin: '10px' }}>
10       <h2>Conditional Rendering</h2>
11       <button onClick={() => setIsLoggedIn(!isLoggedIn)}>
12         {isLoggedIn ? 'Logout' : 'Login'}
13       </button>
14       {isLoggedIn ? (
15         <h3 style={{ color: 'green' }}>Welcome User!</h3>
16       ) : (
17         <h3 style={{ color: 'red' }}>Please Login</h3>
18       )}
19       <p>Score: {score}</p>
20       {score >= 60 ? (
21         <p style={{ color: 'green' }}>Passed! ☑</p>
22       ) : (
23         <p style={{ color: 'red' }}>Failed! ☐</p>
24       )}
25     </div>
26   );
27 }
28 export default ConditionalRender;
```

3.6 String Literals

Use template literals for dynamic text.

Listing 11: StringLiterals.js

```
1 import React from 'react';
2
3 function StringLiterals() {
4   const name = "John Doe";
5   const age = 20;
6   const course = "B.Tech Computer Science";
7
8   const greeting = 'Hello, my name is ${name}';
9   const bio = 'I am ${age} years old studying ${course}';
10  const birthYear = 'I was born in ${new Date().getFullYear() - age}';
11
12  return (
13    <div style={{ padding: '20px', border: '1px solid #ccc',
14      margin: '10px' }}>
15      <h2>String Literals Demo</h2>
```



```
15         <p>{greeting}</p>
16         <p>{bio}</p>
17         <p>{birthYear}</p>
18     </div>
19 );
20 }
21 export default StringLiterals;
```

4 Advanced React Concepts

4.1 useState Hook

Manage state in functional components.

Listing 12: UseStateCounter.js

```
1 import React, { useState } from 'react';
2
3 function UseStateCounter() {
4     const [count, setCount] = useState(0);
5     const [name, setName] = useState('');
6
7     return (
8         <div style={{ padding: '20px', border: '1px solid #ccc',
9             margin: '10px' }}>
10             <h2>useState Hook Demo</h2>
11             <div>
12                 <h3>Counter: {count}</h3>
13                 <button onClick={() => setCount(count + 1)}>Increment
14                     </button>
15                 <button onClick={() => setCount(count - 1)}>Decrement
16                     </button>
17                 <button onClick={() => setCount(0)}>Reset</button>
18             </div>
19             <div style={{ marginTop: '20px' }}>
20                 <h3>Name: {name}</h3>
21                 <input
22                     type="text"
23                     value={name}
24                     onChange={(e) => setName(e.target.value)}
25                     placeholder="Enter your name"
26                 />
27             </div>
28         </div>
29     );
30 }
31 export default UseStateCounter;
```

4.2 useEffect Hook

Handle side effects like API calls.

Listing 13: UseEffectAPI.js

```
1 import React, { useState, useEffect } from 'react';
2
3 function UseEffectAPI() {
4   const [users, setUsers] = useState([]);
5   const [loading, setLoading] = useState(true);
6
7   useEffect(() => {
8     fetch('https://jsonplaceholder.typicode.com/users')
9       .then(response => response.json())
10      .then(data => {
11        setUsers(data);
12        setLoading(false);
13      })
14      .catch(error => {
15        console.error('Error fetching data:', error);
16        setLoading(false);
17      });
18   }, []);
19
20   return (
21     <div style={{ padding: '20px', border: '1px solid #ccc',
22       margin: '10px' }}>
23       <h2>useEffect Hook - API Data Fetching</h2>
24       {loading ? (
25         <p>Loading users...</p>
26       ) : (
27         <div>
28           <h3>Users List:</h3>
29           <ul>
30             {users.map(user => (
31               <li key={user.id}>
32                 {user.name} - {user.email}
33               </li>
34             ))}
35           </ul>
36         </div>
37       )}
38     </div>
39   );
40 }
41 export default UseEffectAPI;
```

4.3 Props for Data Sharing

Pass data between components using props.

Listing 14: ParentComponent.js

```
1 import React from 'react';
2 import ChildComponent from './ChildComponent';
3
```

```
4 function ParentComponent() {
5   const user = {
6     name: "Alice",
7     age: 22,
8     course: "B.Tech IT"
9   };
10  const grades = [85, 90, 78, 92, 88];
11
12  return (
13    <div style={{ padding: '20px', border: '1px solid #ccc',
14      margin: '10px' }}>
15      <h2>Parent Component</h2>
16      <p>This is the parent component passing data to child</p>
17      <ChildComponent
18        user={user}
19        grades={grades}
20        message="Hello from Parent!"
21        isStudent={true}
22      />
23    </div>
24  );
25 }
export default ParentComponent;
```

Listing 15: ChildComponent.js

```
1 import React from 'react';
2
3 function ChildComponent(props) {
4   const { user, grades, message, isStudent } = props;
5
6   return (
7     <div style={{ padding: '15px', border: '1px solid blue',
8       margin: '10px' }}>
9       <h3>Child Component</h3>
10      <p>{message}</p>
11      <h4>User Information:</h4>
12      <p>Name: {user.name}</p>
13      <p>Age: {user.age}</p>
14      <p>Course: {user.course}</p>
15      <p>Status: {isStudent ? 'Student' : 'Not a student'}</p>
16      <h4>Grades:</h4>
17      <ul>
18        {grades.map((grade, index) => (
19          <li key={index}>Subject {index + 1}: {grade}%</li>
20        ))}
21      </ul>
22      <p>Average: {grades.reduce((a, b) => a + b, 0) / grades.
23        length}%</p>
24    </div>
25  );
26 }
```

```
24 }  
25 export default ChildComponent;
```

5 Additional Resources

- **MDN Web Docs** for JavaScript: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- **React.js Documentation**: <https://react.dev>
- **Code Sandbox**: Online coding environment for testing.
- **Vite**: Fast development server for React (<https://vite.dev>).

6 Conclusion

This workshop introduced JavaScript for DOM manipulation and React.js for building dynamic web applications. Practice the examples provided and explore the resources to deepen your understanding. For further assistance, contact your instructor.